



**UNIVERSIDAD SAN FRANCISCO DE QUITO
COLEGIO DE CIENCIAS E INGENIERÍAS**

Semester: 202610

HOMEWORK 01

TOPIC:

Search Methods, Games and Heuristics

DESCRIPTION:

In this assignment you will become familiar with search algorithms, different heuristic strategies and the relationship they have with the evaluation of games.

RESEARCH QUESTION:

1. Chess vs. Go:

In class we talked about games and, in particular, we mentioned two of them: chess and go. In both games, at different points in history, computers appeared that were trained to play them. In this research exercise, investigate what the state of the art is for both chess and go, which algorithm or algorithms are the most advanced ones to play/train them, compare what you find for each game and answer: what is similar?, what is different?, what is the computational complexity of chess?, what is the computational complexity of go?, is it possible to use the classic BFS and DFS algorithms (or their maximization-minimization versions – minimax or alpha-beta pruning)?

EXERCISES:

1. 8-Tile:

The 8-tile is a classic game that consists of eight tiles on a 3x3 board where each tile has a number from one to 8 associated with it and where there is a blank (empty) space. The board allows each tile to be slid from left to right, from top to bottom and vice versa.

The idea of this game is to reach different board configurations, whether you start from a completely ordered board or from a random one.

Your job is to generate the tree (or graph) of possible moves on this board in order to find the path of moves that leads from an initial state of the board to the final (goal) state of the board. To do this, you will explore different search strategies such as: breadth-first search, depth-first search and best-first search (the basis of the A* algorithm). For the best-first search case, explore three heuristics: Euclidean distance, Manhattan distance and a heuristic of your choice (it could be another metric; look up existing metrics and pick one).

- a. Breadth-first search.
- b. Depth-first search.
- c. Best-first search
 - i. Euclidean Distance
 - ii. Manhattan Distance
 - iii. Free Heuristic

2. 80-Tile:

For a moment, let us think about a new version of the previous game, the 80-tile, much harder than the 8-tile. It is the same original idea of the game, but now on a 9x9 matrix



(something similar to Sudoku). Each tile is numbered from 1 to 9 but with a surname A, B, C, D, E, F, G, H, or I (as in Fig. 01) depending on the quadrant where the tile is located. Only quadrant I has an empty space, so tile 9H does not exist.

A	B	C
D	E	F
G	H	I

Fig 01. Quadrants (initial state) of the 80-tile

Your job is to achieve the smallest number of moves (shortest path) while exploring the smallest possible number of states, using one or several heuristics (check the course textbook for ideas), in order to reach the configuration of Fig. 02 (take into account that each quadrant must be ordered as shown in Fig. 03 and that it starts as shown in that same figure).

A	B	C
H	I	D
G	F	E

Fig 02. Final State of the 80-tile

1A	2A	3A		1A	2A	3A
4A	5A	6A	→	8A	9A	4A
7A	8A	9A		7A	6A	5A

Fig 03. Initial and final configuration of each quadrant of the 80-tile

Show a portion of the explored tree, the total number of states generated, the total number of states analyzed and the length of the path found.

3. 8-Queens.

The 8-queens problem on a chessboard (8x8) is a classic computing problem and it is usually used to test the concept of backtracking.

This time, using an imperative language (C++, Java, C, Python, etc.), write a program that solves the generic problem of placing N (chess) queens on a board of dimensions NxN.

4. Farmer, Wolf, Goat, and Cabbage Problem:

The Farmer, Wolf, Goat, and Cabbage Problem is a famous logic puzzle. A farmer must transport a wolf, a goat, and a cabbage from one side of a river to the other. The farmer has a boat that can carry only himself and one of the three items at a time. The farmer must always be in the boat when it crosses the river. However, if the farmer leaves the wolf alone with the goat on either bank, the wolf will eat the goat. Similarly, if the farmer leaves the goat alone with the cabbage, the goat will eat the cabbage. The goal is to determine a sequence of crossings that allows the farmer to transport himself, the wolf, the goat, and the cabbage safely to the opposite bank.

Your job is to model this problem and find the set of steps that do not violate the constraint, so that the farmer can cross from one bank of the river to the other.

5. Missionaries and Cannibals Problem:

The missionaries and cannibals problem is another logic puzzle that poses a challenge similar to the Farmer, Wolf, Cabbage Problem. This problem involves three missionaries and three cannibals who are on one side of a river and must cross it using a boat with a maximum capacity of two people.



However, there is an important constraint: on either side of the river, if the missionaries are outnumbered by the cannibals, the cannibals will eat the missionaries.

Your job is to model this problem and find the set of steps that do not violate the constraint, so that the three cannibals and the three missionaries can successfully cross from one bank of the river to the other.

6. **Sudoku:** Sudoku is a constraint satisfaction puzzle played on a 9x9 grid that is divided into nine 3x3 boxes. Some cells arrive already filled in (the givens) and the rest are empty. The goal is to write a digit from 1 to 9 in every empty cell so that no digit is repeated in any row, in any column, or in any of the nine 3x3 boxes. A well-posed Sudoku has exactly one solution.

Your job is to write a program, in an imperative language (C++, Java, C, Python, etc.), that solves any 9x9 Sudoku board using backtracking, the same technique you used for the N-Queens exercise. Model the puzzle as a constraint satisfaction problem: the variables are the empty cells, the domain of every variable is the set {1, 2, ..., 9}, and the constraints state that the nine values of every row, of every column, and of every 3x3 box must all be different.

The basic algorithm is the one you already know: choose an empty cell, try each value that is consistent with the current partial board, recurse over the remaining empty cells and, if the recursion fails, undo the assignment (backtrack) and try the next value. If no value works for a cell, the failure is propagated back to the previous cell.

Solve at least the two boards below. The board in Fig. 04 is an easy/medium board with 30 givens; the board in Fig. 05 is a hard board with only 22 givens, deliberately built so that the naive version of the algorithm is forced into a very large number of backtracks. Your program must read the board from a text file (0 or a dot for an empty cell) so that it can also be tested with other boards.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Fig 04. Board 1 (30 givens)

8	5				2	4		
7	2							9
		4						
			1		7			2
3		5				9		
	4							
				8			7	
	1	7						
				3	6		4	

Fig 05. Board 2 (22 givens)

Besides the naive version, implement and compare at least two of the following improvements over plain backtracking:

- a. Variable ordering with the Minimum Remaining Values (MRV) heuristic: always assign first the empty cell that has the fewest legal values left.
- b. Value ordering with the Least Constraining Value heuristic: try first the value that removes the fewest options from the neighboring cells.



- c. Constraint propagation: forward checking after every assignment, and/or the classic naked single and hidden single rules.

For every board and for every version of the algorithm, report: the number of assignments (states generated), the number of backtracks, the maximum recursion depth reached and the execution time. Present these numbers in a comparison table and explain why the heuristics change them the way they do.

Finally, answer the following questions: how many boards would a brute force generate-and-test strategy have to examine in the worst case, and why does backtracking behave so much better than that? Also, modify your program so that instead of stopping at the first solution it counts every solution of a board, and use it to verify that the two boards above have exactly one solution.

DELIVERABLES:

For this assignment you will submit:

1. A pdf with the research exercise plus the relevant comments about the practical exercises.
2. In addition, for the practical exercises, you will include the source code of the solution you developed.
3. Finally, remember that this assignment has two parts: solving the exercises and documenting them (report – 20%) and the in-person defense of this work (80%) during the professor's office hours within one week after the assignment due date.
4. Remember that the use of Generative AI is strictly restricted to the conversational (chat) version of the model of your preference. If you use Generative AI, you must submit the machine-readable transcript of that interaction with your chat; if the transcript is not submitted, your grade will be deeply affected. The use of AI agents is forbidden for any task; if agents are used, you will receive a grade of 0 on this assignment.

QUESTIONS AND INQUIRIES:

If you have questions or inquiries, do not hesitate to write to the e-mail: drifrioa@usfq.edu.ec

If you have urgent questions, you can write to me on Telegram under the username: @danielrioerio